Assignment 4

aob85593@uga.edu

Experiment 1:

Algorithm	Input Type	Comparisons	Big-O-Notes
Selection Sort	Ordered	49995000	O(n^2) its always N(N-1)/2 regardless of input
Selection Sort	Random	49995000	O(N^2) identical
Selection Sort	Reverse	49995000	O(N^2), also identical
Merge Sort	Ordered	69008	O(N log N) low because of the ordered file
Merge Sort	Random	120414	O(N log N), because of the increase of element crossing more comparisons are needed
Merge Sort	Reverse	64608	O(N log N), lowest because the reversed pairs merge easily
Heap Sort	Ordered	244460	O(N log N), an ordered input forces more heap fixes
Heap Sort	Random	235430	O(N log N), heapify depth drives count nt input order
Heap Sort	Reverse	226682	O (N log N); reverse is already heap-like after build
QuickSort-fp	Ordered	49995000	O(N^2) is the worst case
QuickSort-fp	Random	162640	O(N log N) random input avoids worst case
QuickSort-fp	Reverse	49995000	O(N^2) worst case - first pivot on reversed = always largest
QuickSort-rp	Ordered	~159100	O(N log N) - random pivot avoids worst case
QuickSort-rp	Random	~150900	O(N log N) - random pivot performs similarly

QuickSort-rp	Reverse	~159400	O(N log N) the random pivot avoids the worst case

Q I — Selection Sort input sensitivity:

Selection sort always does exactly N(N-1)/2 = 49,995,000 comparisons regardless of input order. This is consistent with $O(N^2)$ theory the inner loop always runs the same number of times because there is no early termination. Input type has zero effect.

Q II — Merge Sort: random (~120,414) is ~double sorted (~64,608–69,008):

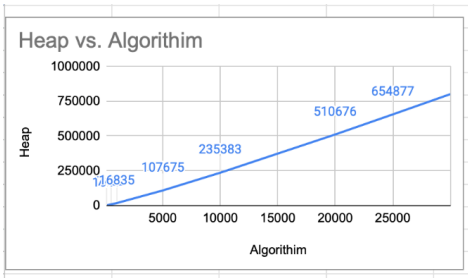
In a sorted or reversed run, many merge steps end early because all remaining elements on one side are already smaller/larger than everything on the other side, the merge finishes after only one sub-array is exhausted. In the random case both sub-arrays are interleaved, so comparisons continue longer through nearly every merge call, roughly doubling the total count.

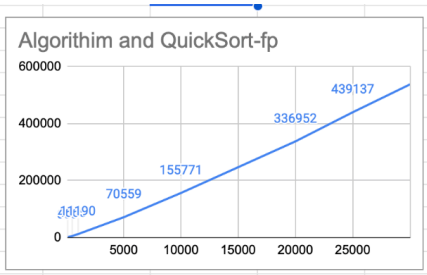
Q III — Heap Sort stability across inputs (~226,682–244,460):

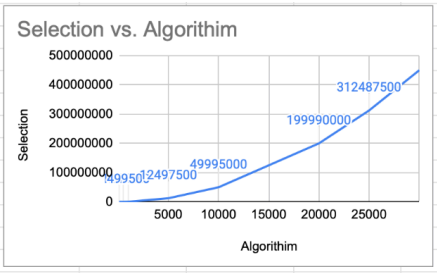
Heap sort's comparison count is driven by the tree structure, not by the original input order. The build-heap and extract phases visit nodes based on tree shape, which is fixed by N alone. The ordered file is slightly higher than reversed because building the heap from an ascending array requires more sift-down operations

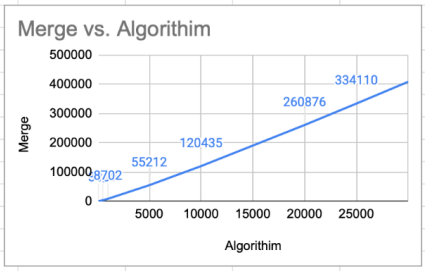
Experiment 2:

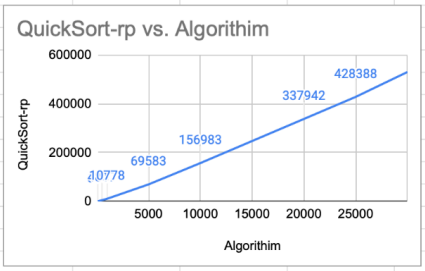
Algorithim	100	500	1000	5000	10000	20000	25000	30000
Merge	542	3859	8702	55212	120435	260876	334110	408569
Heap	1029	7432	16835	107675	235383	510676	654877	800666
QuickSort-fp	653	4665	11190	70559	155771	336952	439137	536965
QuickSort-rp	644	4817	10778	69583	156983	337942	428388	530524
Selection	4950	124750	499500	12497500	49995000	199990000	312487500	449985000











Experiment 2 Discussion:

Selection Sort: Results confirm $O(N^2)$ behavior, growing from 4,950 at n=100 to 449,985,000 at n=30,000.

Merge Sort: Results confirm O(N log N) behavior, growing from 542 at n=100 to 408,569 at n=30,000.

Heap Sort: Results confirm O(N log N), growing from 1,029 to 800,666. Heap sort consistently produces more comparisons than merge sort due to larger constant factors from sift-down operations, but the overall curve matches theory.

QuickSort-fp: Results show O(N log N) for random inputs, from 653 to 536,965. This matches theory for random data, though performance would degrade to $O(N^2)$ on ordered or reversed inputs.

QuickSort-rp: Results confirm O(N log N), from 644 to 530,524. Values mirror QuickSort-fp since both receive random input, and the plot aligns with the theoretical N log N curve.